Components Overview (Stage 2)

Container Diagram

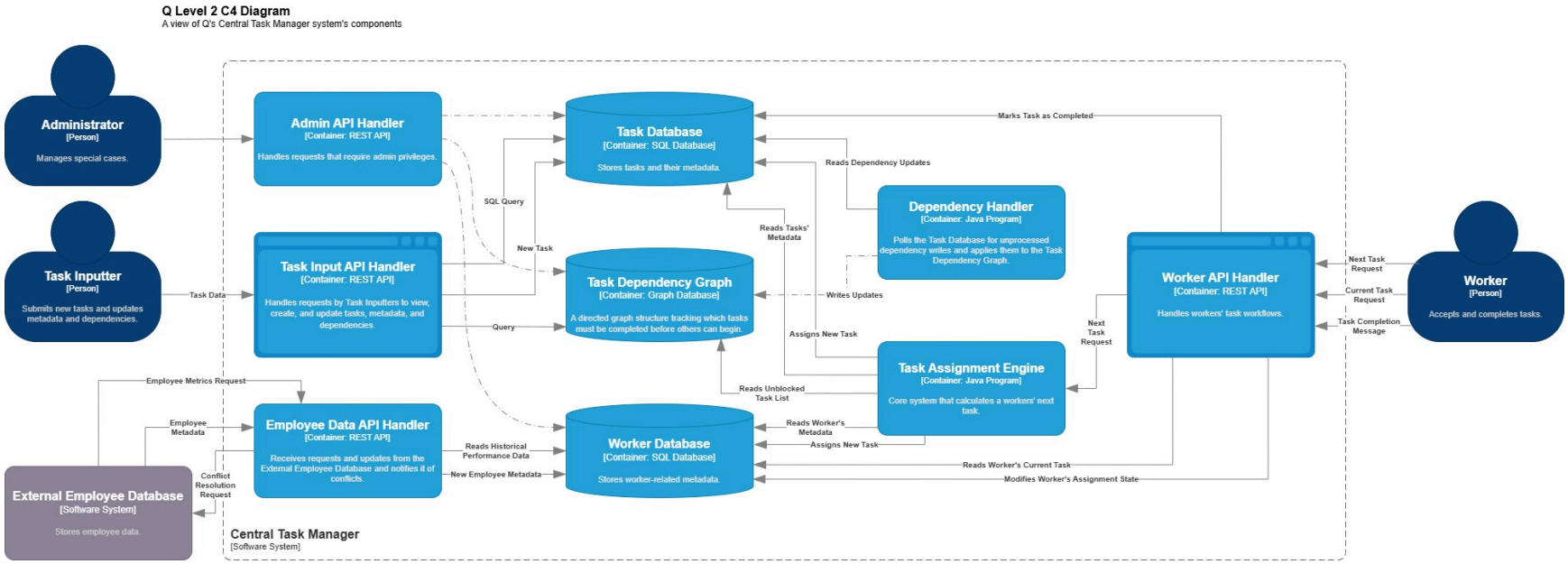


Figure 2

As demonstrated in this diagram, the central task management system is composed of the following parts:

- Task-related Databases (Task Database and Task Dependency Graph), which store data submitted by Task Inputters. The Task Database maintains the state of each task.
- Dependency Handler, which ensures that the Task Dependency Graph stays up-to-date.
- Worker Database, which stores data received from the External Employee Database, as well as each worker's task history. It also maintains the state of each worker.

- Task Assignment Engine, the central calculation system.
- API Handlers for each of the parties listed in the System Overview: Data Sources (Task Inputters and External Employee Database), Workers, and Administrators.

Four Main Workflows

Workflow 1 - Task Inputter submits new data

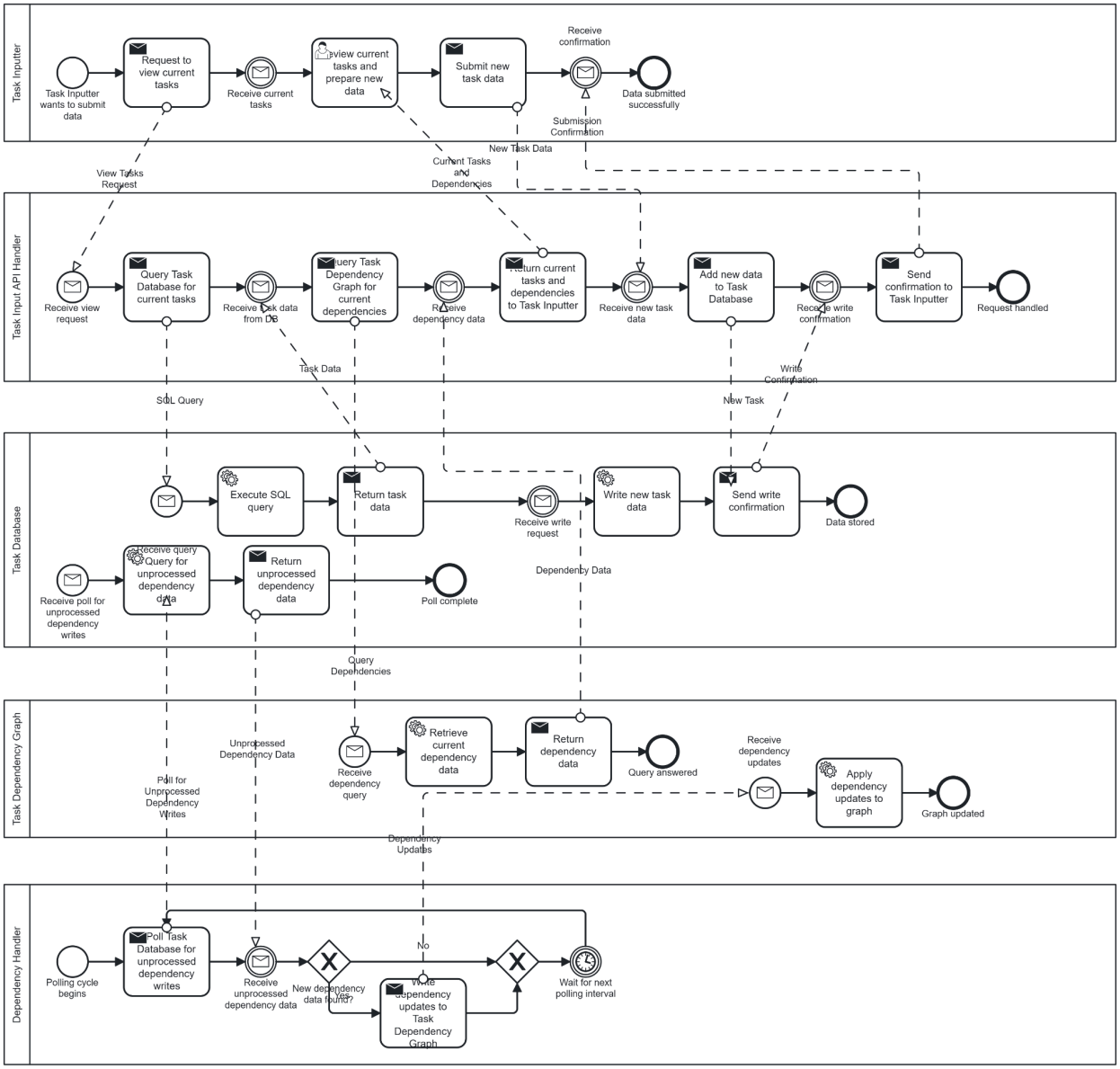


Figure 3

1. A Task Inputter views the current tasks, which the Task Input API Handler fetches from the Task Database.
2. The Task Inputter adds new data, which the Task Input API Handler adds to the Task Database.
3. The Dependency Handler extracts any changes in dependency data and notifies the Task Dependency Graph of these changes.

Failure Scenario:

- It is possible for the Task Input API Handler to fail after adding the task to the Task Database. This will mean that the Task Inputter will be unable to ensure the data was inputted successfully, but they can check the Task Database afterwards to find out.

Workflow 2 - Worker requests a new task

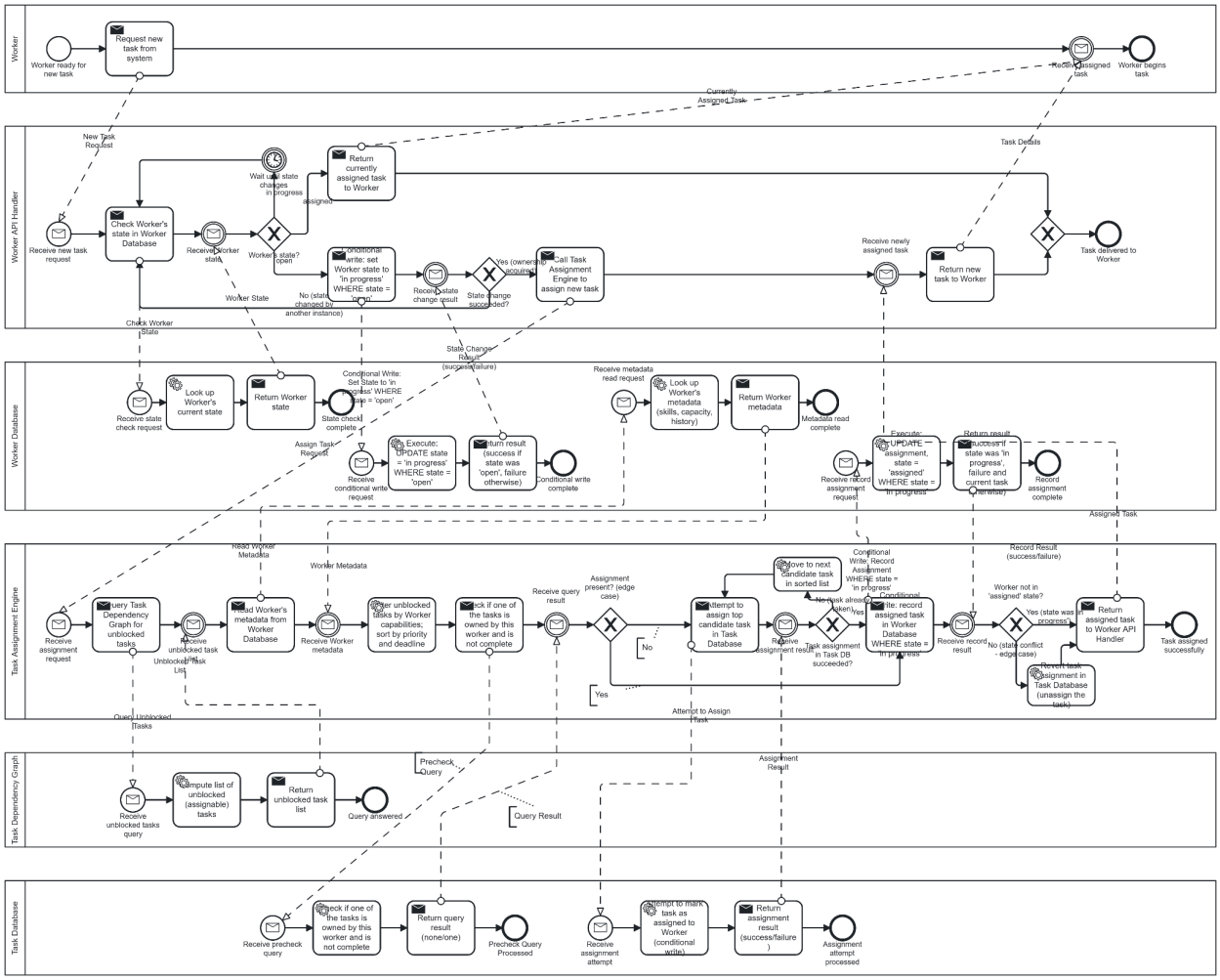


Figure 4

1. A Worker requests a new task.
2. The Worker API Handler checks the Worker Database for the Worker's state. If the state is 'in progress', it waits until the state changes. If the state is 'assigned', it returns the assigned task. If the state is 'open', it changes the Worker's state in the Worker Database to 'in progress', and calls the Task Assignment Engine to assign a new task, returning the new task upon assignment.
3. If called, the Task Assignment Engine first queries the Task Dependency Graph for unblocked tasks and filters this list with the data stored in the Worker Database to obtain a list of the Worker's potential tasks, which it sorts by priority and deadline.

4. The Task Assignment Engine then attempts to assign the top result to the Worker in the Task Database, proceeding to the next one in the list upon a failure (indicating that the task has been assigned to a different Worker).
5. Upon success, the Task Assignment Engine adds the task to the Worker in the Worker Database and marks the Worker's state as 'assigned'.

Failure Scenarios:

- The Worker API Handler or Task Assignment Engine can fail mid-assignment. Therefore, the state of 'in progress' can time out and revert to 'open', allowing the worker to be obtained by a different Task Assignment Engine instance. To ensure this doesn't cause conflicts between two instances, the Task Assignment Engine must ensure that the state is not 'assigned' before modifying the worker.
- It is possible for all unblocked tasks to be assigned. In that case, the Task Assignment Engine will mark the Worker's state as 'open' and return that there were no available tasks.
- The Task Assignment Engine may fail after assigning the task in the Task Database but before assigning it to the worker in the Worker Database. This is okay because the worker will eventually request a new task again. The Task Dependency Graph is assignment-agnostic and so will still return that previously-assigned task in its list of tasks. However, the problem is that a new task may have superseded that one in priority in the meantime, meaning the Task Assignment Engine won't look at the previously-assigned task and miss it. Therefore, the Task Assignment Engine checks all the tasks it receives from the Task Dependency Graph to see if they are already owned by this Worker before trying to assign a new one. There is a loophole to this if the Worker stops working, but this edge case is an acknowledged risk and can be handled by admins.

Workflow 3 - Worker marks their task as complete

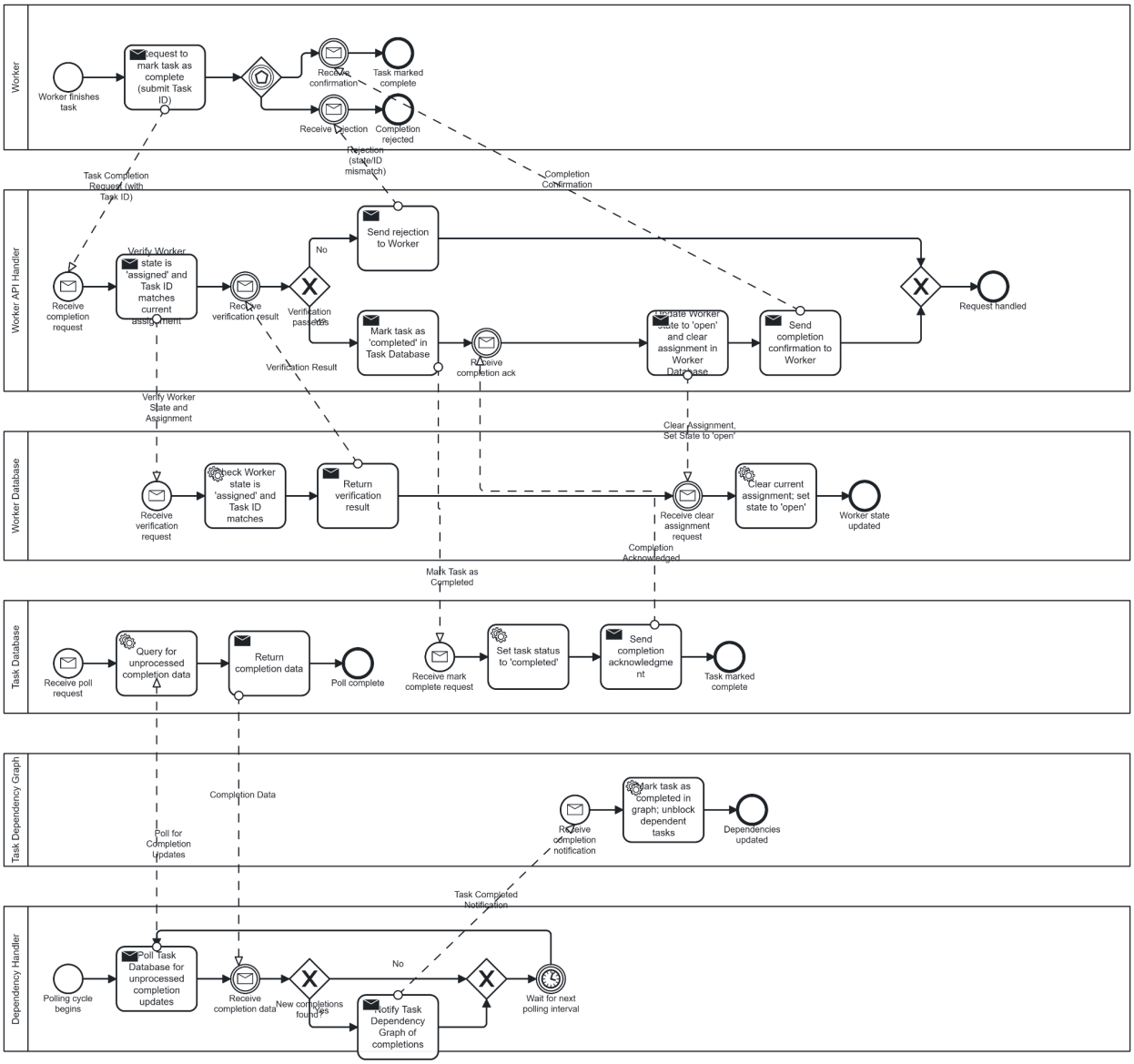


Figure 5

1. A Worker requests to mark their current task as complete.
2. The Worker API Handler verifies that the Worker's state is 'assigned' and that the submitted task matches the Worker's current assignment in the Worker Database.
3. The Worker API Handler marks the task as 'completed' in the Task Database.

4. The Worker API Handler updates the Worker’s state to ‘open’ and clears the Worker’s current assignment (if that task is still the worker’s assignment) in the Worker Database.
5. The Dependency Handler notices the update and notifies the Task Dependency Graph of the task's completion.

Failure Scenario:

- The Worker API Handler can fail after marking the task as ‘completed’ and before updating the Worker’s status, but this can be remedied by having the Worker retry sending their completion message.

Workflow 4 - Administrator modifies data

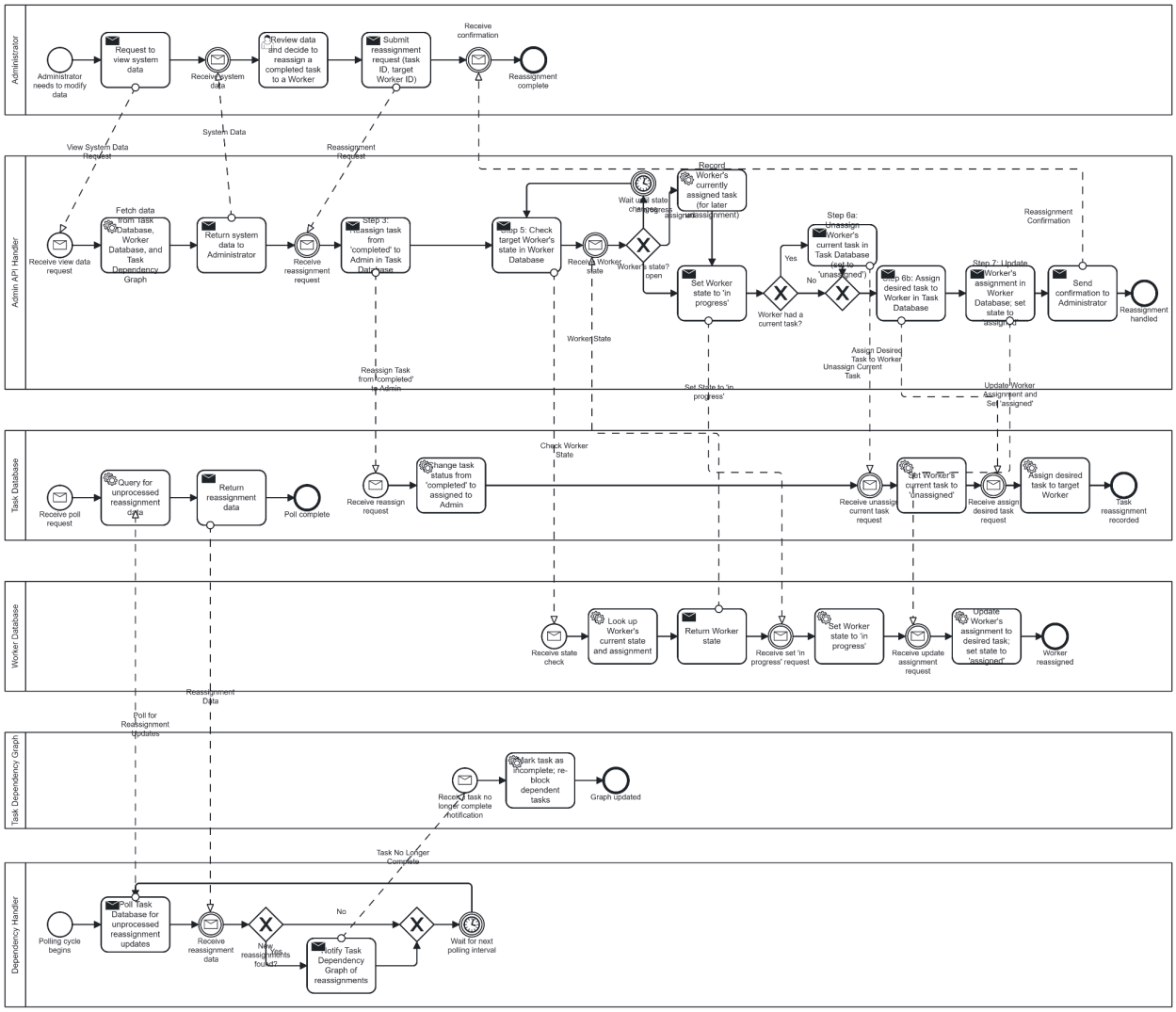


Figure 6

1. An Administrator views the data in the system, which the Admin API Handler fetches as needed from the Task and Worker Databases.
2. The Administrator decides that a completed task is unfinished and requests to reassign the task to a Worker who is currently working on a different task.
3. The Admin API Handler reassigns the desired task in the Task Database from ‘completed’ to the Administrator.
4. The Dependency Handler notices that this task is no longer complete and updates Task Dependency Graph to block accordingly.

5. The Admin API Handler checks the Worker's state in the Worker Database. If the state is 'in progress', the Admin API Handler waits until the state changes. If the state is 'open', the Admin API Handler changes the state to 'in progress'. If it is 'assigned', the Admin API Handler records the currently assigned task and then changes the state to 'in progress'. (Unlike in Workflow 2, this 'in progress' will not time out.)
6. The Admin API Handler reassigns the intended Worker's currently assigned task, if it exists, in the Task Database to be 'unassigned'.
7. The Admin API Handler assigns the desired task to the Worker.
8. The Admin API Handler marks the Worker's state in the Worker Database as 'assigned'.

Failure Scenario:

- If the Admin API Handler fails after locking the Worker, the Worker will be stuck and require an Administrator to unlock them.